

Software Architectures for Autonomous Robotics: The ROS 2 Middleware

NECKER

June 15, 2026

Contents

1	Introduction to a Robot's Software Architecture	2
1.1	Hardware-Software Decomposition: TurtleBot3 Case Study	2
1.2	The Concept of Middleware and Abstraction	3
2	The ROS 2 Framework (Robot Operating System)	3
2.1	ROS 2 Distributions and Release Cycle	5
3	The ROS 2 Computational Graph	5
3.1	Interfaces and the IDL Language	5
3.2	Nodes	5
3.3	Managed Nodes (Managed Lifecycles)	5
3.4	Topics: Publish/Subscribe Mechanism	7
3.5	Services: Request/Response Structure	7
3.6	Actions: Structure based on Goal, Feedback, and Result	7
3.7	Parameters	7
3.8	Anatomy of Common Sensory Messages	7
3.8.1	LaserScan (<code>sensor_msgs/msg/LaserScan</code>)	8
3.8.2	Images (<code>sensor_msgs/msg/Image</code>)	8
4	The TF2 Subsystem (Spatial Transformations)	9
5	The DDS Middleware and Quality of Service (QoS)	9
5.1	DDS Decentralized Architecture	10
5.2	Quality of Service Profiles (QoS)	10
6	Behavior Trees	10
6.1	The "Tick" Mechanism and Return States	11
6.2	Node Taxonomy in Behavior Trees	11
6.2.1	Control Nodes	11
6.2.2	Decorator Nodes	11
6.2.3	Leaf Nodes	11
6.3	Analysis of an Advanced Decision Workflow	12
6.3.1	Execution Loop Analysis (<i>Tree Trace</i>)	12

1 Introduction to a Robot's Software Architecture

From an engineering and computer science perspective, an autonomous robot cannot be considered merely a simple computer, but rather an integrated complex cyber-physical system, equipped with sensors (for environmental perception) and actuators (for physical interaction). The design of control software requires the coexistence of pure high-intensity algorithmic computational tasks with strict constraints of real-time performance, determinism, and reactivity to environmental stimuli.

1.1 Hardware-Software Decomposition: TurtleBot3 Case Study

To understand the need for a modular and layered software architecture, it is helpful to analyze the task decomposition within an academic and industrial reference platform like the **TurtleBot3 Burger**. The management of computational resources is distributed across an eterogeneous hardware topology, divided primarily between high-level processing and low-level real-time execution:

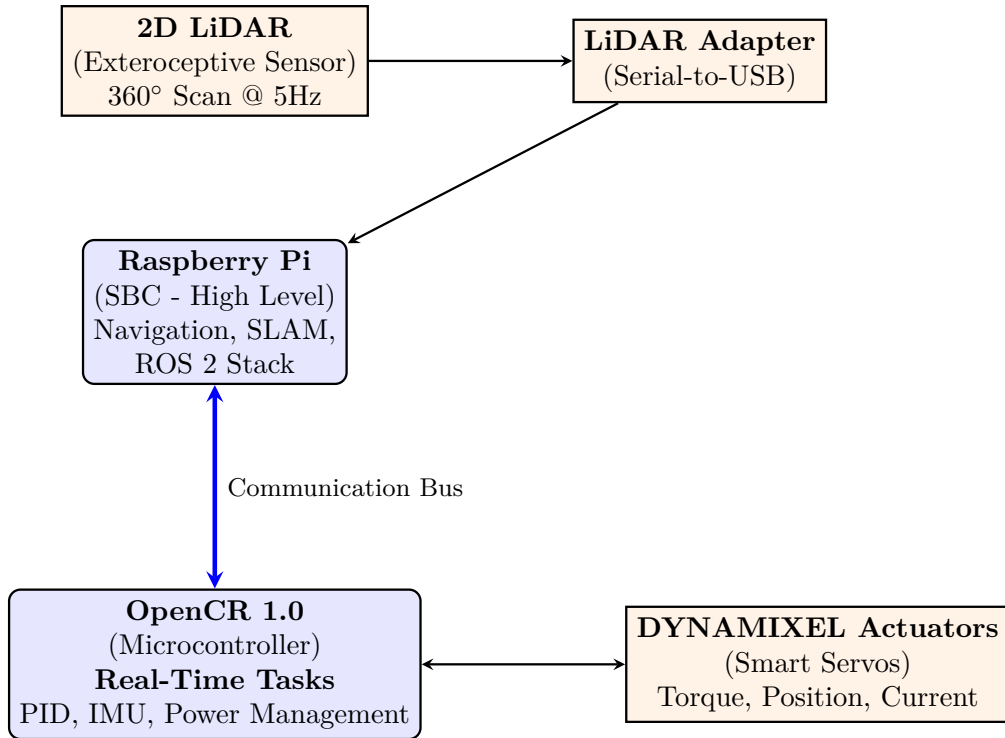


Figure 1: Hardware topology and computational partitioning of the TurtleBot3 Burger.

- **Single-Board Computer (Raspberry Pi):** Dedicated to tasks with high computational intensity and not strictly bound by deterministic real-time constraints. It handles the processing of LiDAR point clouds, the execution of *Simultaneous Localization and Mapping* (SLAM) algorithms, global and local trajectory planning (*MoveBase/Nav2*), and the orchestration of the middleware nodes.
- **Specialized Microcontroller (OpenCR 1.0):** An ARM Cortex-M7 architecture-based board specifically designed for ROS. It handles very low-latency tasks that require temporal determinism, including: the PID control loop for motor speed, the reading of raw data from the Inertial Measurement Unit (6-axis IMU with gyroscope and accelerometer), battery status diagnostics, and the direct hardware interface.

- **2D LiDAR Sensor:** An exteroceptive sensor that performs a continuous planar scan at 5Hz within a 360° radius up to a distance of 3.5 meters, converting raw serial data via a USB adapter to make it accessible to the Single-Board Computer.
- **DYNAMIXEL Actuators:** Smart servomotors equipped with an internal microprocessor, capable of returning precise feedback on angular position, speed, temperature, and current load to the microcontroller.

1.2 The Concept of Middleware and Abstraction

To prevent the developer from having to rewrite low-level drivers and manually manage thread synchronization and network packet serialization, modern software architectures introduce a **Middleware** layer. The middleware sits as a software abstraction above the operating system and the hardware, standardizing access to sensors/actuators and formalizing data exchange channels between distinct processes through a data-centric (*data-centric*) paradigm.

2 The ROS 2 Framework (Robot Operating System)

ROS 2 represents a radical evolution of ROS 1, redesigned from scratch to satisfy industrial requirements that the original framework could not inherently guarantee, such as: support for *bare-metal* embedded systems, real-time compliance (*Real-Time compliance*), tolerance to unreliable or bandwidth-limited wireless networks, cryptographic security of the data channel, and multi-robot architectures.

The internal architecture of ROS 2 is structured as a strictly layered stack designed to maximize portability and decouple the application interface from physical transport:

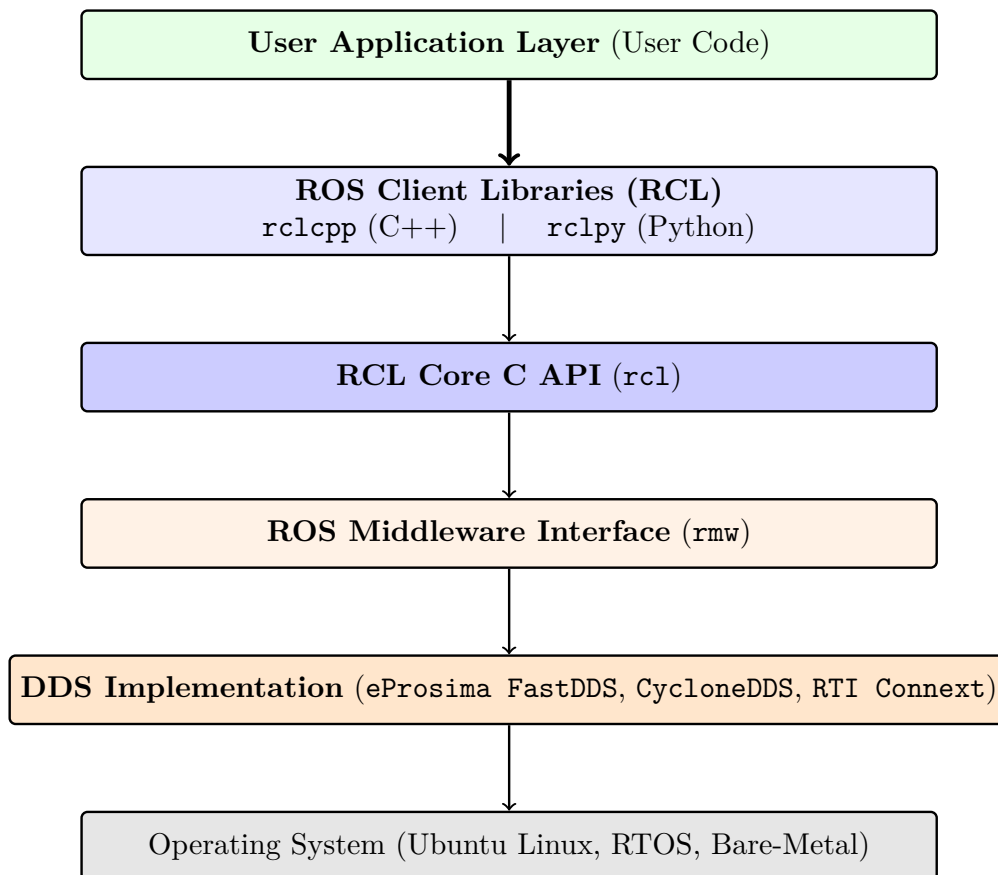


Figure 2: Internal layered architecture of the ROS 2 framework.

- **User Application Layer (User Code):** This is the highest level and represents the source code written by the developer. The specific logic of the robot resides in this layer, such as navigation algorithms, motor control, or image processing, implemented by exploiting the abstract concepts (nodes, topics, services) provided by the framework.
- **ROS Client Libraries (RCL):** These are the front-end libraries specific to the programming language chosen by the user. The most common are `rclcpp` for C++ and `rclpy` for Python. They provide a rich and idiomatic high-level interface (API) to interact with the ROS 2 ecosystem, making development more intuitive.
- **RCL Core C API (rcl):** This is the common core of the framework, written in pure C language. It implements the foundational logic of ROS 2 in a language-neutral manner. This layer ensures that the behavior of the system (such as timer management or the execution loop) remains consistent and identical, whether the user is programming in Python, C++, or Rust.
- **ROS Middleware Interface (rmw):** This is a crucial abstraction layer (*wrapper*) that separates ROS 2 logic from the underlying network implementation. Thanks to this interface, ROS 2 is middleware-agnostic: it allows the user to switch the underlying DDS vendor without changing a single line of application code.
- **DDS Implementation:** Represents the actual implementation of the standard network protocol *Data Distribution Service* (DDS). This layer manages the "dirty work" of network communication: automatic node discovery (*peer-to-peer discovery*), data serialization/deserialization, and the enforcement of Quality of Service (QoS) policies. The user can choose among various supported vendors, such as `eProsima FastDDS`, `Eclipse CycloneDDS`, or `RTI Connext`.
- **Operating System:** This is the host operating system on which the entire stack runs. In addition to standard support for Ubuntu Linux (the de facto standard in robotics), the modular architecture of ROS 2 and the use of DDS enable native execution on macOS, Windows, real-time operating systems (RTOS), and embedded *Bare-Metal* platforms lacking a traditional operating system.

As we can see, **ROS 2** is considered a **middleware** and not a program for several reasons, such as:

- It is not a single program, but each node is a different program so that the entire system does not crash together if something fails.
- **ROS 2** is bound to a specific version of Linux due to its dependencies on **glibc** libraries that change with each Linux release.
- A program generally does not integrate an entire package management system, which **ROS 2** does.

2.1 ROS 2 Distributions and Release Cycle

Unlike traditional software, ROS is distributed in thematic packages called "Distributions" (*Distributions*). Each distribution ensures that a specific set of modules and libraries is compatible and tested to work together.

Distribution names follow an alphabetical order and are composed of an adjective and the name of a turtle species (e.g., *Ardent Apalone*, *Bouncy Bolson*, *Humble Hawksbill*). A key feature of the ROS 2 architecture is that each distribution is strictly tied to a specific release of the base operating system, typically Ubuntu Linux. For example, the **Humble Hawksbill** distribution (released in May 2022) is anchored to Ubuntu 22.04 LTS. This link ensures stability and prevents conflicts with system libraries. Furthermore, there is a special distribution called **Rolling Ridley**, which is constantly updated and serves as a testing area (*staging*) for future stable releases.

3 The ROS 2 Computational Graph

The Computational Graph is the peer-to-peer network of robotic processes that jointly process data. The atomic elements composing this graph are structured according to formal communication rules.

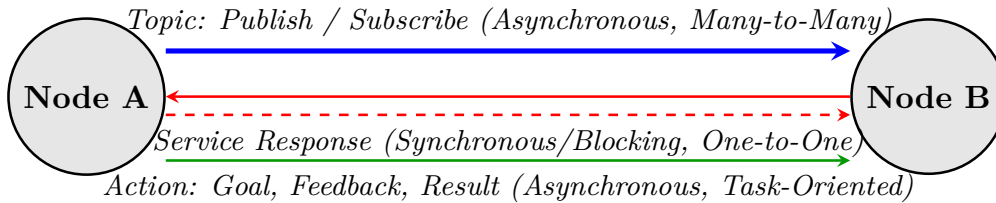


Figure 3: The three main communication paradigms in the ROS 2 Computational Graph.

3.1 Interfaces and the IDL Language

For nodes to interact by exchanging data, ROS 2 enforces the use of strict **Interfaces**. An interface acts as a formal "contract" ensuring that the sending node and the receiving node perfectly agree on the structure and type of data exchanged. These contracts are written in a neutral language called **IDL (Interface Definition Language)**, which allows nodes developed in different programming languages (e.g., C++ and Python) to communicate without any compatibility issues. At the file system level, interfaces are divided into three extensions: `.msg` for Topic messages, `.srv` for Service requests/responses, and `.action` for complex Action tasks.

3.2 Nodes

A **Node** is the elementary computational unit responsible for a specific and isolated task (e.g., motor control, laser filtering). In ROS 2, node composition strictly follows the object-oriented programming (OOP) paradigm. A node is implemented by inheriting from the base class `rclcpp::Node`, encapsulating publishers, subscribers, services, and timers inside it as private class members, guaranteeing clean code and memory isolation.

3.3 Managed Nodes (Managed Lifecycles)

In the standard launch of a node, it begins executing its behavior as soon as it is started (the *spin* method). However, for complex control systems, this approach is limiting. To overcome this issue, ROS 2 introduced **Managed Lifecycles** (Managed nodes).

"Managed" nodes transition through a deterministic state machine composed of four main states: **Unconfigured**, **Inactive**, **Active**, and **Finalized**. This paradigm enables an extremely reliable startup (*bringup*) of the robot: it guarantees that all critical dependencies and hardware interfaces are correctly loaded and initialized before the node starts processing operational data. An external supervisor can pause, restart, or shut down a single node at runtime without having to tear down or restart the entire computational graph.

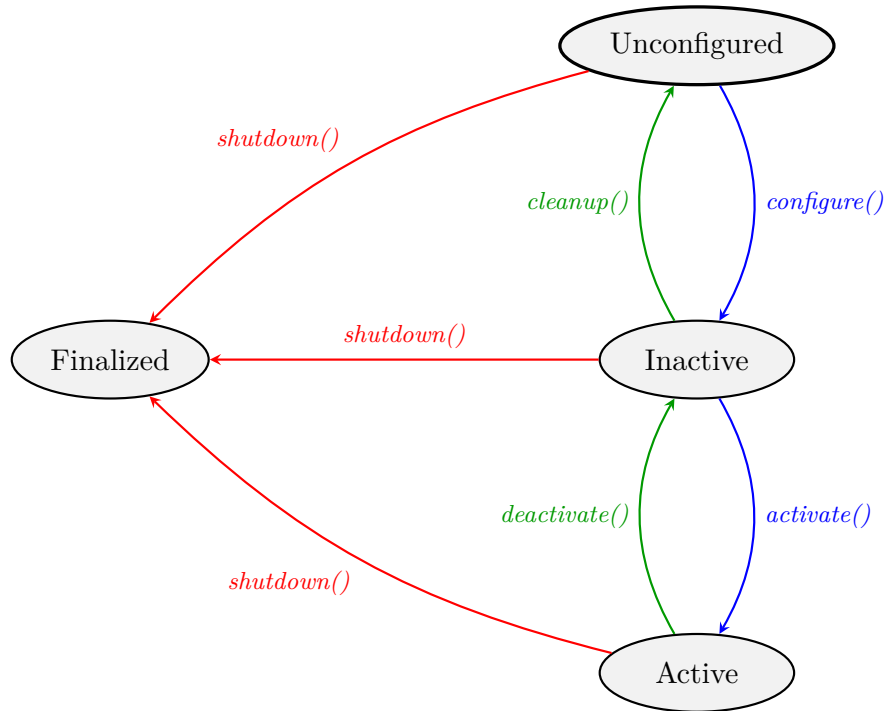


Figure 4: Simplified state machine of the Lifecycle Node in ROS 2. Transitions allow deterministic life cycle control.

WARNING: the **Finalized** state corresponds to the destruction of the node; in fact, if one were to simply switch it to **Unconfigured** to deactivate it, the node would still remain active in **RAM**. Performing a shutdown operation, instead, triggers a routine to safely delete the node, preventing, for example, a motor from remaining active because the last sent command did not tell it to stop.

3.4 Topics: Publish/Subscribe Mechanism

Topics implement an asynchronous, anonymous, and unidirectional communication channel of the *many-to-many* type.

- **Mechanism:** A node that produces data publishes (*Publish*) it to a specific topic; one or more nodes interested in that data stream subscribe (*Subscribe*) to the same topic. Nodes do not know each other's identities; they only know the name and type of the channel.
- **Ideal use cases:** Continuous data streams from sensors at a constant frequency, where occasional packet loss is preferable to a system block (e.g., LiDAR readings, odometry coordinates, images coming from a camera).

3.5 Services: Request/Response Structure

Services (*Services*) define a bidirectional, *one-to-one* communication modality of the client-server type.

- **Mechanism:** A Server node offers a specific functionality. A Client node sends a *Request* (data request) and waits in a synchronous or asynchronous blocking manner until the server processes the computation and returns a *Response* (response).
- **Ideal use cases:** One-off operations, sudden mathematical computations, reset commands, or configuration change requests (e.g., taking a high-resolution photograph, zeroing odometry, requesting the static map of the environment).

3.6 Actions: Structure based on Goal, Feedback, and Result

Actions (*Actions*) represent the most complex communication paradigm in ROS 2, designed specifically to coordinate long-term and evolving tasks. An action relies on an asynchronous non-blocking client-server structure that internally implements three distinct communication channels (created by combining underlying topics and services):

1. **Goal Channel (Service):** The client sends a request to start a complex task (e.g., "navigate to point *X*"). The server accepts or rejects the goal.
2. **Feedback Channel (Topic):** During task execution, the server sends periodic asynchronous updates to the client to monitor progress (e.g., "remaining distance to target: 1.2 meters").
3. **Result Channel (Service):** Upon execution completion, the server notifies the final state of the operation (success, failure, canceled) by sending the final summary data.

A fundamental advantage of actions is **preemptability**: the client has the option to send a cancellation command (*Cancel Goal*) at any time during execution if external conditions require it, safely interrupting the task.

3.7 Parameters

Parameters in ROS 2 are configuration variables structured as key-value pairs (e.g., `max_velocity = 0.22`) associated with a single node. They allow changing the dynamic and algorithmic behavior of the software at execution time (*Runtime*) without having to recompile the source code, exposing a dynamic reconfiguration interface.

3.8 Anatomy of Common Sensory Messages

To standardize environmental data processing, ROS 2 provides packages containing pre-built message definitions. Two of the most widely used in autonomous navigation are the LiDAR and the camera.

3.8.1 LaserScan (`sensor_msgs/msg/LaserScan`)

Represents a planar scan performed by a laser rangefinder. The message is composed of:

- **Header:** Contains the `timestamp` (acquisition time) and the `frame_id` (the coordinate frame of the sensor).
- **Angular Geometry:** The `angle_min` and `angle_max` fields define the scan opening in radians (e.g., from 0 to 2π for a 360° coverage). The `angle_increment` field indicates the angular resolution between two contiguous measurements.
- **Distances (`ranges`):** A dynamic floating-point array containing the physical measurements of distance in meters for each scanned angle. Values outside hardware limits (`range_min` and `range_max`) are discarded.
- **Intensity (`intensities`):** Indicates the strength of the reflected beam, useful for understanding the reflectivity or material of the hit obstacle.

3.8.2 Images (`sensor_msgs/msg/Image`)

Used to transmit uncompressed pixel matrices.

- The `height` and `width` fields define the spatial dimensions of the image in rows and columns.
- The `encoding` field is a critical string that tells the Subscriber how to interpret the raw data (e.g., `rgb8` for 8-bit color images, or `32FC1` for floating-point depth maps).
- The `data` array contains the continuous sequence of image bytes.

Due to their size, the continuous streaming of uncompressed images rapidly saturates network bandwidth. For this reason, ROS 2 utilizes a plugin system called **image_transport**, which allows publishing images by applying compression algorithms (such as JPEG, PNG, or Theora video codecs) leveraging differences between consecutive frames.

4 The TF2 Subsystem (Spatial Transformations)

A robot is physically composed of multiple moving parts and sensors mounted in asymmetrical positions. ROS 2 handles the geometric relationships between all these parts through a unified approach called the **TF2** subsystem.

TF2 keeps track over time of the homogeneous coordinates and rototranslation matrices (transformations) that link the various reference systems (called *frames*). The system adopts very strict physical conventions:

- It follows the **right-hand rule** (Right-handed) to define the X, Y, Z axes.
- Rotation angles increase counter-clockwise (CCW): zero (0) points forward, $\frac{\pi}{2}$ points left, $-\frac{\pi}{2}$ points right, and $\pm\pi$ points backward.

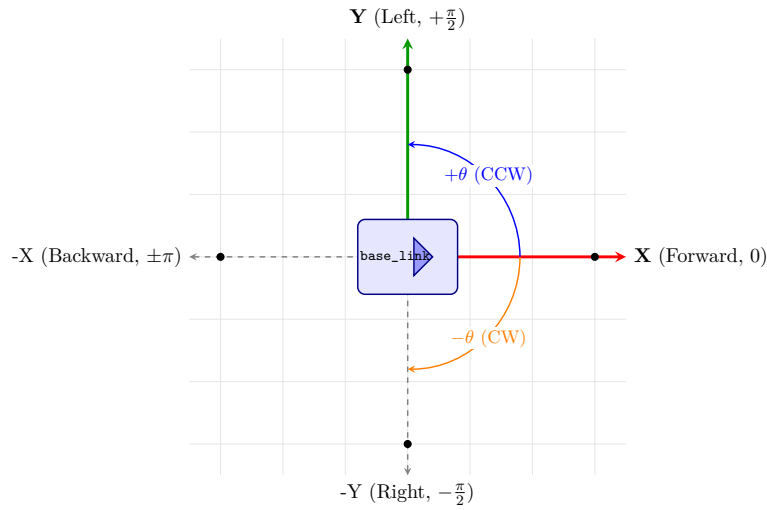


Figure 5: Standard ROS 2 spatial convention (TF2) based on the right-hand rule. The view is *Top-Down*. The X-axis indicates the front of the robot, the Y-axis the left flank. Rotations occur around the Z-axis (pointing out of the page) and increase counter-clockwise.

This transformation tree relies on two architectural topics to manage the diversity of the physical world:

1. **/tf**: Used for dynamic transformations with high temporal volatility, such as moving wheels or the joints of a robotic arm. This data expires rapidly if not constantly updated. **WARNING: TFs are always expressed relative to the coordinate system of the parent frame and not the base_link.**
2. **/tf_static**: Reserved for rigid and immutable transformations, for instance, the position where the LiDAR sensor is screwed relative to the geometric center of the robot (**base_link**). Having no expiration time, this information is transmitted only once over the network, adopting a *Transient Local* Quality of Service (QoS).

5 The DDS Middleware and Quality of Service (QoS)

Unlike **ROS 1**, in **ROS 2** the centralized **ROS Master** has been removed in favor of adopting the industrial standard **DDS (Data Distribution Service)**.

5.1 DDS Decentralized Architecture

DDS is a *data-centric* communication protocol standardized by the OMG (*Object Management Group*), widely used in the aerospace, military, and industrial automation sectors. Thanks to DDS, ROS 2 becomes a completely decentralized and peer-to-peer system: each node is capable of discovering the presence of other nodes on the same local network autonomously via **UDP Discovery** packets, eliminating the *Single Point of Failure* represented by the old ROS 1 Master.

5.2 Quality of Service Profiles (QoS)

DDS allows fine configuration of message transmission modes at the network transport level through **Quality of Service (QoS)** policies. Properly setting these rules prevents network congestion and guarantees system robustness. The four fundamental policies are:

QoS Policy	Available Options	Mechanical Behavior
History	- Keep Last (with depth N) - Keep All	Keep Last stores in memory only the last N received messages. Keep All stores every message until memory is exhausted.
Reliability	- Reliable - Best Effort	Reliable implements error control (ACK/NACK): if a packet is lost, it is resent. Best Effort sends data without confirmation, tolerating losses.
Durability	- Volatile - Transient Local	Volatile erases past messages; new subscribers only see future messages. Transient Local requires the publisher to store the latest messages to send them to nodes that subscribe late.
Liveliness	- Automatic - Manual By Node	Defines how nodes assert their presence on the network to prove they are still active and functioning (<i>heartbeat</i>).

Table 1: Summary of foundational QoS policies in the DDS-based ROS 2 middleware.

Some examples of how to use them:

- **Configuration for High-Frequency Sensors (LiDAR, Cam):** Requires **Reliability: Best Effort** and **History: Keep Last** (depth=1). It is not useful to overload the network to recover an old laser scan; it is critical to have instantaneous access to the latest produced reading.
- **Configuration for Static Maps or Critical Coordinates:** Requires **Reliability: Reliable** and **Durability: Transient Local**. If a navigation node starts up late, it must be able to immediately receive the entire environment map published hours earlier by the mapping node.

6 Behavior Trees

In complex autonomous robotics, high-level decision planning (*Action Selection*) requires a flexible software architecture. Historically, this task was assigned to **Finite State Machines (FSM)**. However, FSMs suffer from state explosion, poor modularity, and structural rigidity: adding a state requires manually redesigning transitions to all other pre-existing states. To overcome these limitations, modern navigation stacks (such as *Nav2* in ROS 2) adopt **Behavior Trees (BT)**.

6.1 The "Tick" Mechanism and Return States

A Behavior Tree is a directed acyclic graph structured as a hierarchical tree. Control execution is driven by a periodic signal emitted from the root (*Root Node*) called a **Tick**. The Tick propagates from top to bottom and from left to right along the branches of the tree.

Every time a node receives a Tick, it executes its internal code and must **instantaneously** return one of three possible cognitive status signals to its parent node:

- **SUCCESS:** The node has completed its task successfully.
- **FAILURE:** The node has failed to execute the task assigned to it.
- **RUNNING:** The task is being executed but requires time to complete (typical of physical robot actions). At the next cycle, the Tick will return to query this node.

6.2 Node Taxonomy in Behavior Trees

Nodes in a Behavior Tree are strictly divided into two macro-categories: internal nodes (control or decorator nodes) and leaf nodes (execution nodes).

6.2.1 Control Nodes

Their purpose is to route the Tick flow to their children based on the received return states.

- **Sequence ($\rightarrow$):** Expresses the logic AND. It executes its children sequentially from left to right. If a child returns **SUCCESS**, the Sequence immediately moves to the next child. If a child returns **FAILURE**, the Sequence aborts the execution of all subsequent children and immediately returns **FAILURE** to its parent. If a child returns **RUNNING**, the Sequence temporarily blocks and returns **RUNNING**.
- **Fallback / Selector ($?$):** Expresses the logic OR (error handling or alternative behaviors). It executes its children from left to right. If a child returns **FAILURE**, the Fallback does not give up; instead, it moves to query the next one to attempt a recovery. If a child returns **SUCCESS**, the Fallback considers itself satisfied, stops scanning, and returns **SUCCESS**. If a child returns **RUNNING**, it returns **RUNNING**.
- **Parallel ($\Rightarrow$):** Sends the Tick to all its children simultaneously, handling the concurrent execution of multiple activities. It returns **SUCCESS** or **FAILURE** based on whether a given percentage or a preset number of children has completed execution.

6.2.2 Decorator Nodes

They can have **only one single child**. Their purpose is to alter or filter the child's return state or condition its Tick (e.g., **Inverter** turns success into failure and vice versa; **RetryUntilSuccessful** re-executes the child N times in case of failure before giving up).

6.2.3 Leaf Nodes

They represent the direct interface between the tree logic and the functional ROS 2 stack. They cannot have children.

- **Condition Node (Ellipse):** Inspects the state of the system or the environment (e.g., checks the battery level or the presence of an obstacle). It returns **SUCCESS** or **FAILURE** immediately; it can never return **RUNNING**.
- **Action Node (Rectangle):** Commands a physical action or a complex computational task prolonged over time (e.g., moving a motor, calculating a path). It typically returns **RUNNING** until the activity is concluded.

6.3 Analysis of an Advanced Decision Workflow

To appreciate the elegance of Behavior Trees in ROS 2 integration, let us analyze the following hierarchical diagram, which models a continuous patrolling loop with a battery safety check and a recovery routine (*Recovery*):

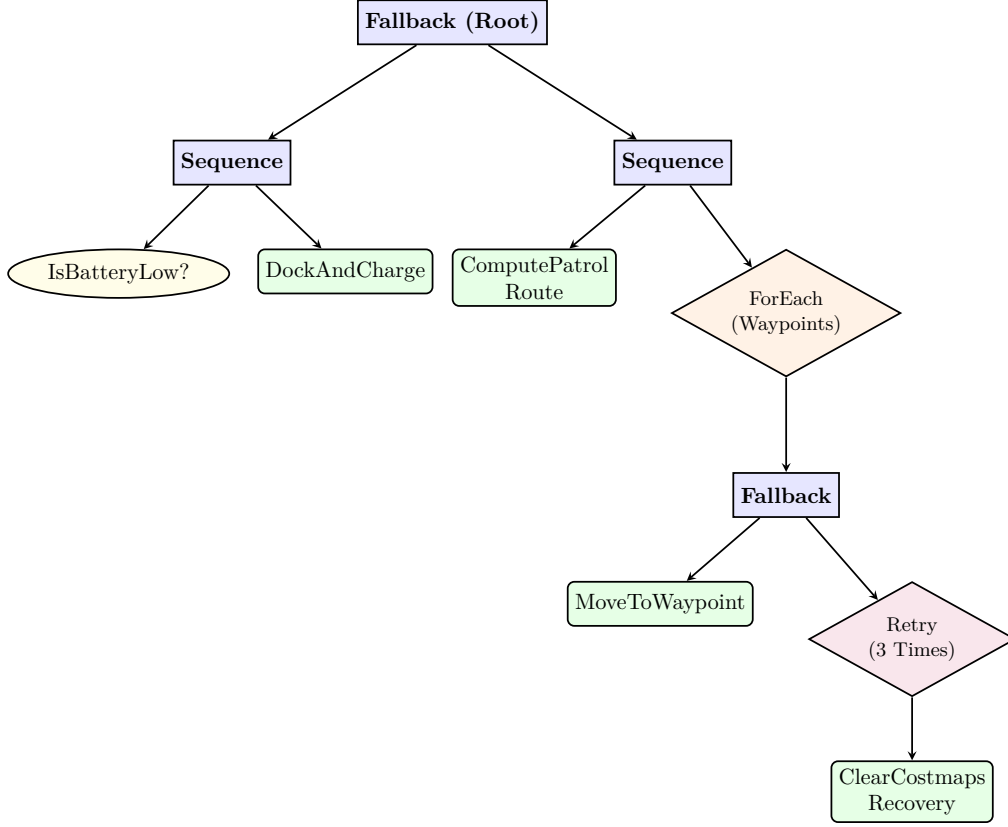


Figure 6: Hierarchical Behavior Tree for autonomous navigation and anomaly management of a patrolling robot.

6.3.1 Execution Loop Analysis (*Tree Trace*)

When the system is running, the main selector (**Fallback Root**) periodically sends the Tick downwards:

1. **Safety Inspection:** The Tick first enters the left branch (**Battery Sequence**). The **IsBatteryLow?** condition analyzes the thermal and charge status.
 - *Case A (Charged Battery):* The condition fails, returning **FAILURE** immediately. The **Sequence** aborts. The parent **Fallback Root** sees a failure on the left branch and, due to its algorithmic nature, **immediately switches to the right branch** to keep the robot operational.
 - *Case B (Low Battery):* The condition succeeds (**SUCCESS**). The **Sequence** proceeds and ticks the **DockAndCharge** action. This action sends a navigation command to the charging station via a ROS 2 Action and returns **RUNNING**. The cycle halts here until the robot is charging, preventing patrolling execution.
2. **Main Task Execution:** In Case A, the Tick flows into the right branch (**Patrol Sequence**). The **ComputePatrolRoute** action is triggered, which computes the patrol points on the map. Once they are obtained (**SUCCESS**), the loop enters the iterative decorator **ForEach**.

3. **Robust Collision Management (Recovery Wrapper):** For each individual waypoint, the underlying **Fallback** attempts the **MoveToWaypoint** action.
- If the robot travels without obstacles, the action returns **RUNNING** and then **SUCCESS** upon arrival. The **Fallback** is satisfied and moves to the next point.
 - If an unexpected obstacle (e.g., a person standing in the hallway) blocks the path, **MoveToWaypoint** fails (**FAILURE**). The **Fallback** does not crash the tree; instead, it routes the Tick to the right recovery branch. The **Retry** decorator triggers the **ClearCostmapsRecovery** action, a ROS 2 service that clears the sensors' local memory to eliminate any ghost readings. This operation is attempted up to 3 times. If the recovery succeeds, at the next cycle **MoveToWaypoint** will be able to recompute the trajectory and complete the task, guaranteeing total autonomy on the field.